Revision — 1.2.2 Applications Generation

OCR H446 — one-page revision summary

Must know

- **Application software** lets the user perform a real-world task (word processor, browser, game). **Utility software** maintains or optimises the computer itself (defrag, backup, antivirus, compression, firewall). *Antivirus is utility, not application.*
- **Open source** software has source code that is **available and modifiable**, usually free with community support (Linux, Firefox, GIMP). **Closed source (proprietary)** hides the source code, usually needs a paid licence and has official vendor support (Windows, Photoshop).
- **Translators** convert code into a form the machine can run. Three types: **assembler**, **compiler**, **interpreter**.
- **Assembler** translates **assembly language into machine code**, roughly **one mnemonic to one machine instruction**.
- **Compiler** translates high-level source into machine code **all at once**, producing a standalone **executable** before the program runs. The program then runs fast, needs no source or translator present, but the executable is platform-specific and all errors are reported together at the end.
- **Interpreter** translates and executes high-level source **one line at a time** at run time; it produces **no executable**, needs the source and interpreter present each run, stops at the **first error**, and is portable and good for developing/debugging.
- **Stages of compilation (in order): lexical analysis → syntax analysis → code generation → code optimisation** (see below). **Syntax errors are reported during syntax analysis.**
- **Library** = a collection of **pre-written, pre-compiled, tested** subroutines that are reused — saving development time, improving reliability and promoting reuse.
- **Linker** combines the compiled program with libraries and other modules into a single executable, resolving references. **Static** linking copies library code into the executable (larger, self-contained); **dynamic** linking keeps it separate (e.g. a **.dll**) and links it at run time (smaller, shared, updatable).
- **Loader** = OS software that copies the executable from storage **into RAM** and prepares it to run.

Key terms

Term	Definition
Application software	Software that lets the user do a real-world task
Utility software	System software that maintains/optimises the computer
Assembler	Translates assembly language into machine code (≈ one-to-one)
Compiler	Translates high-level source into machine code all at once, making an executable
Interpreter	Translates and executes high-level source one line at a time
Library	Pre-written, pre-compiled, tested subroutines reused in programs
Linker	Combines a compiled program with libraries/modules into one executable
Loader	OS software that loads an executable into RAM ready to run

Worked example / how to — the four stages of compilation (in order)

1. **Lexical analysis** — source code is broken into **tokens**; whitespace and comments are removed; a **symbol table** is built.
2. **Syntax analysis (parsing)** — tokens are checked against the language's **grammar** and a **parse tree** is built; **syntax errors are reported here**.
3. **Code generation** — the parse tree and symbol table are used to produce **object/machine code**.
4. **Code optimisation** — the code is refined to **run faster / use less memory** while giving the same result.

Exam tips

- Compare compiler vs interpreter by stating **when** translation happens and **whether an executable is produced** — the *program* runs faster, not the compiling.
- Learn the compilation order and don't confuse **lexical** (tokens) with **syntax** (grammar/errors).
- To protect and distribute finished software, choose a **compiler** (ship the executable only, no source needed); for developing/debugging, an **interpreter** is better.
- Distinguish **static vs dynamic** linking clearly (copied-in vs separate .dll linked at run time), and don't mix up **linker** with **loader**.